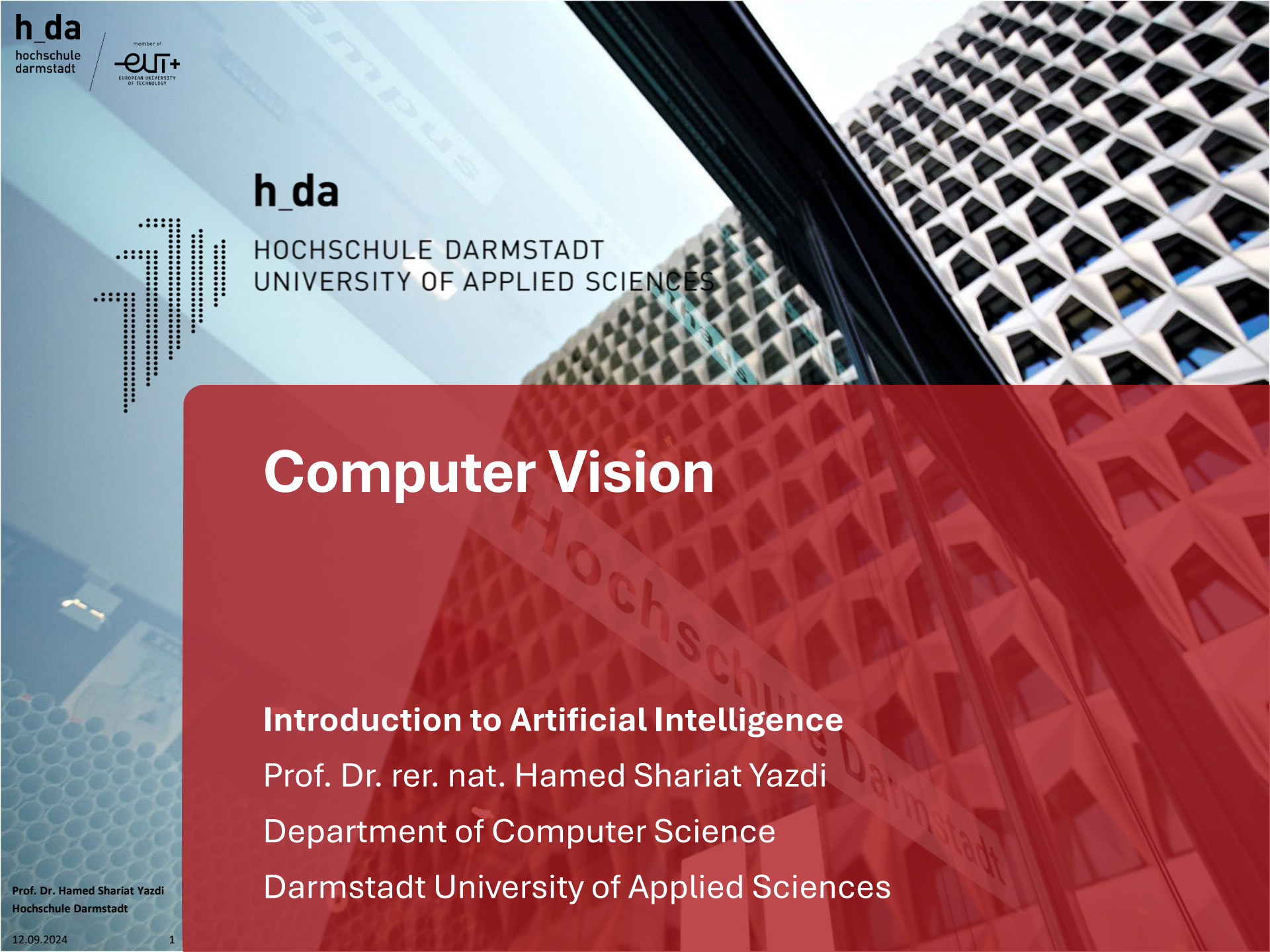


Computer Vision

Introduction to Artificial Intelligence

Prof. Dr. rer. nat. Hamed Shariat Yazdi

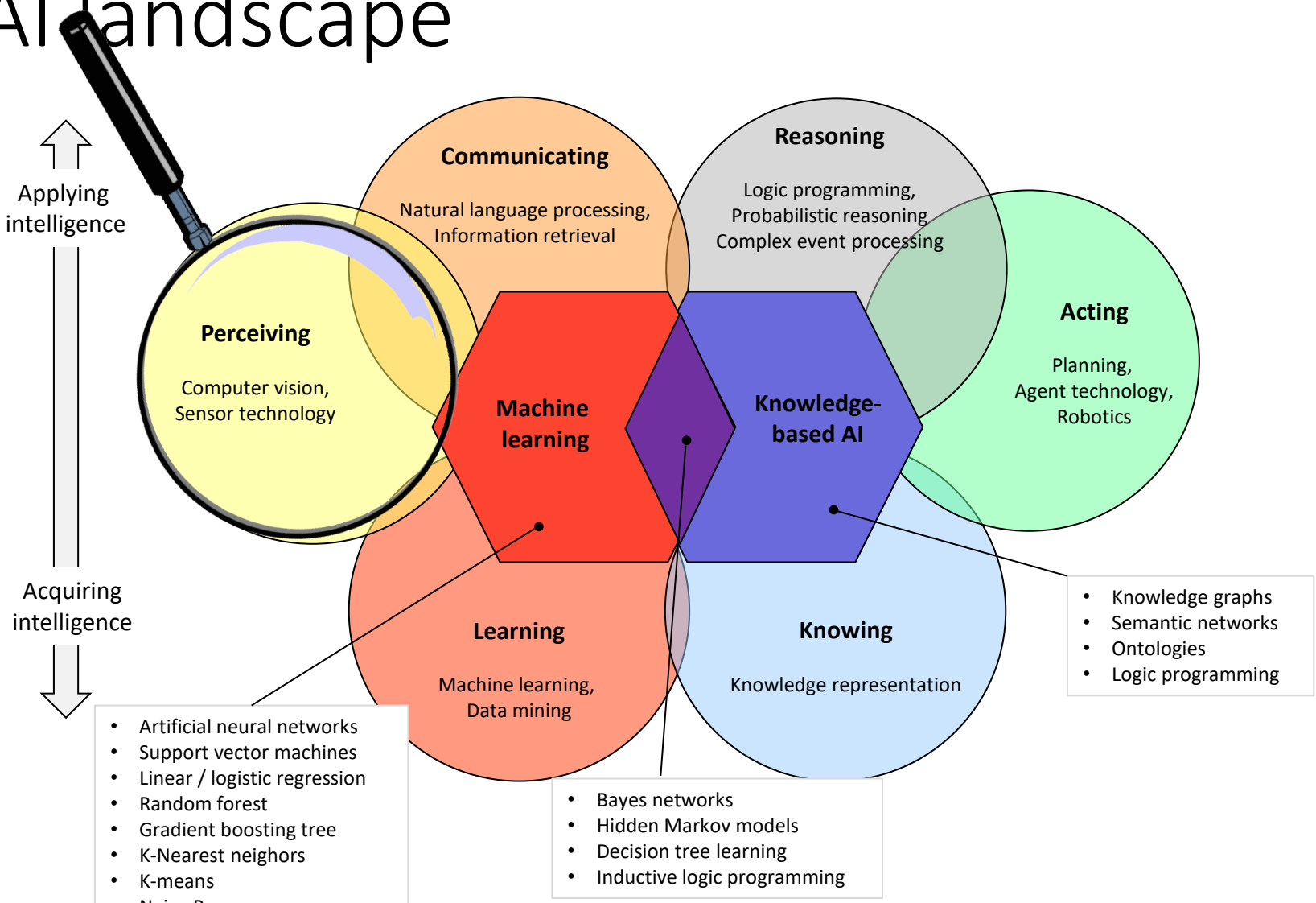
Department of Computer Science

Darmstadt University of Applied Sciences

Agenda

- Overview
- CV Applications
- Deep learning with CNN
- Challenge: Digit recognizer
- Quick check

AI landscape



Inspired by https://www.sigs-datacom.de/order/poster/Landkarte-KI-Poster-AI-2018.php

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

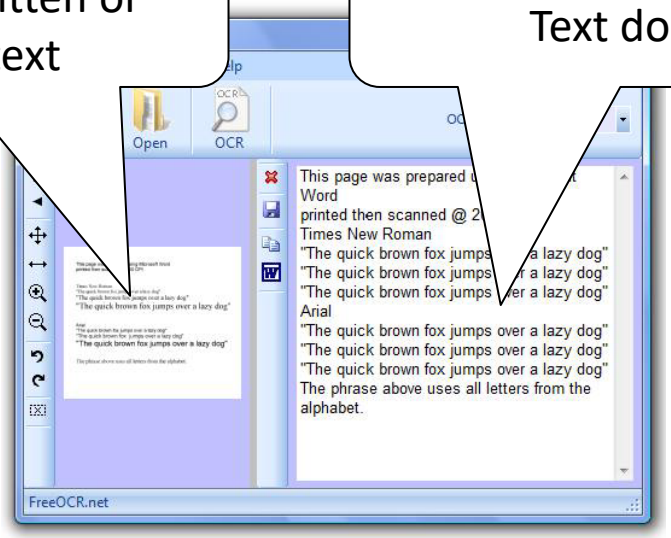
Agenda

- Overview
- CV Applications
- Deep learning with CNN
- Challenge: Digit recognizer
- Quick check

Optical Character Recognition (OCR)

Input:
Image of handwritten or typewritten text

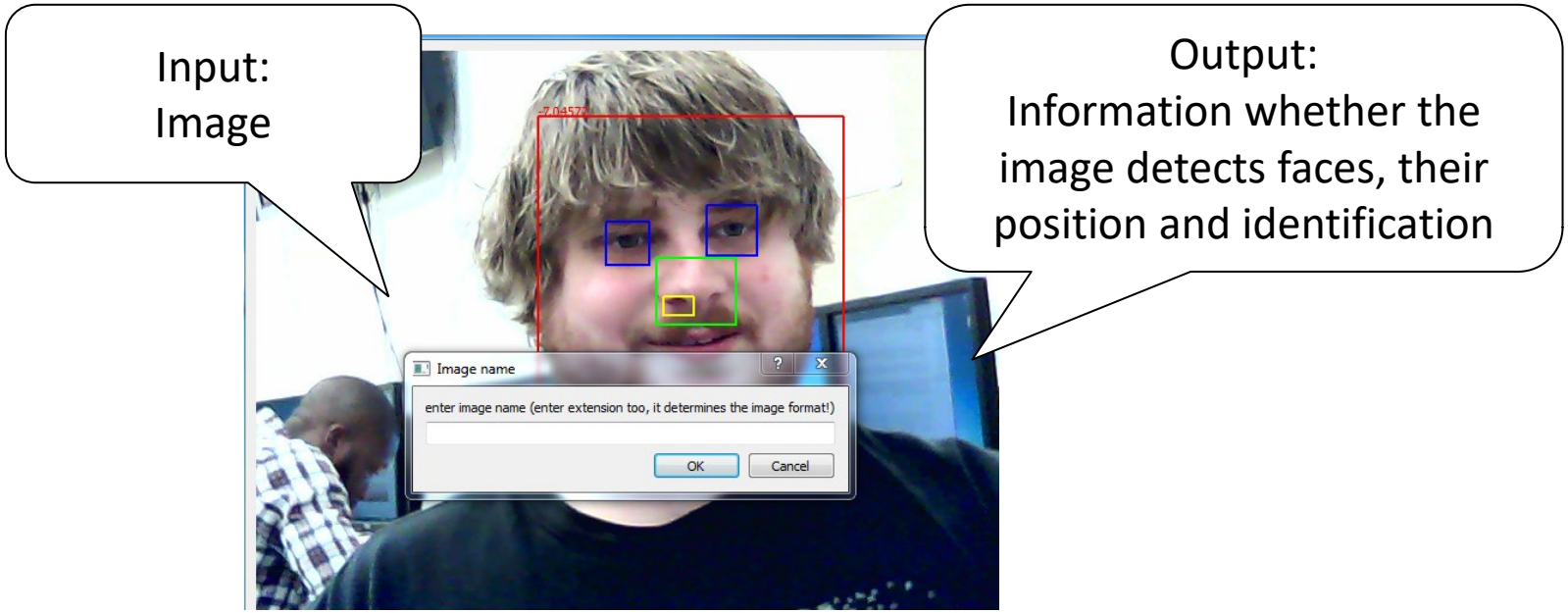
Output:
Text document



- Use Cases:
  - Archiving and retrieving old documents
  - Scanning addresses on letters
  - Evaluating forms
  - …

Source: http://www.freeocr.net/

Face Recognition



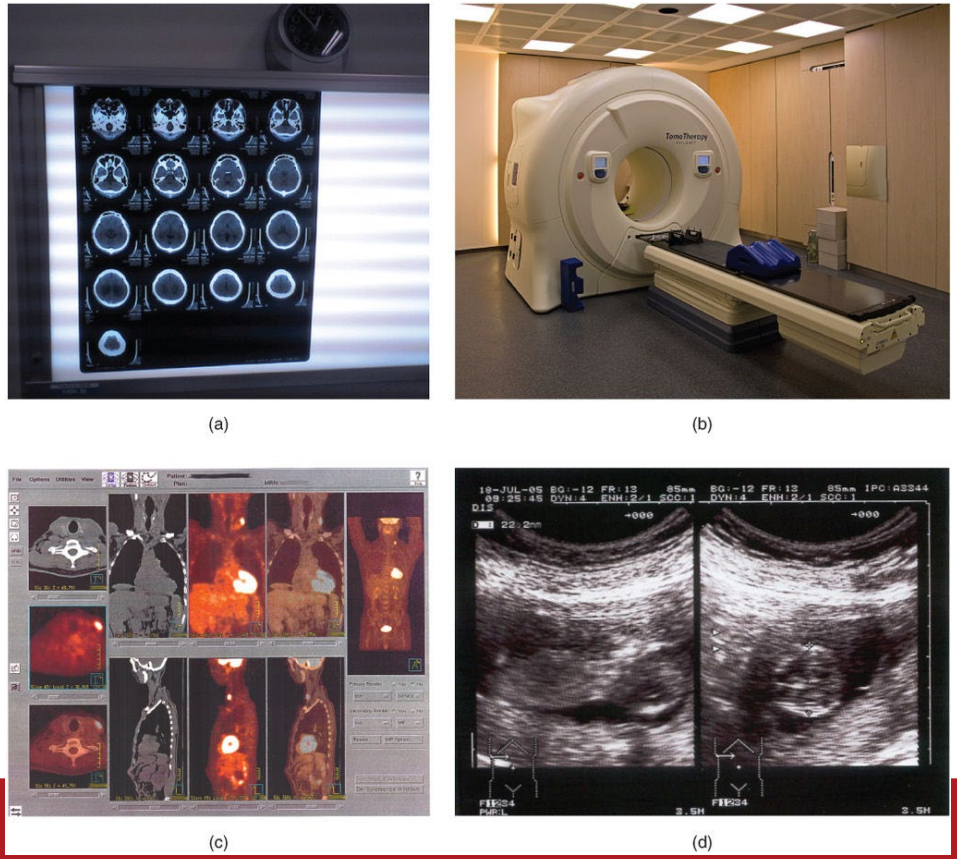
- Use Cases:
  - Autotagging camera images
  - Retrieving Images on the Internet
  - Detecting people (e.g., on airports)
  - …

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Source: Wikimedia

Medical Applications

- Use cases:
  - Detecting anomalies in CT / MRI / Ultrasound images
  - …



Source: Wikipedia

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Industrial / Agricultural Applications

- Use Cases:
  - Sorting fruits in agriculture
  - Quality Management in manufacturing processes
  - Robot control in manufacturing processes
  - …





Source: Wikimedia

Military / Aerospace Applications

- Use Cases:
  - Detection of enemy soldiers or vehicles
  - Missile guidance
  - Autonomous vehicles
  - Drones, …







Source: Wikimedia, Wikipedia

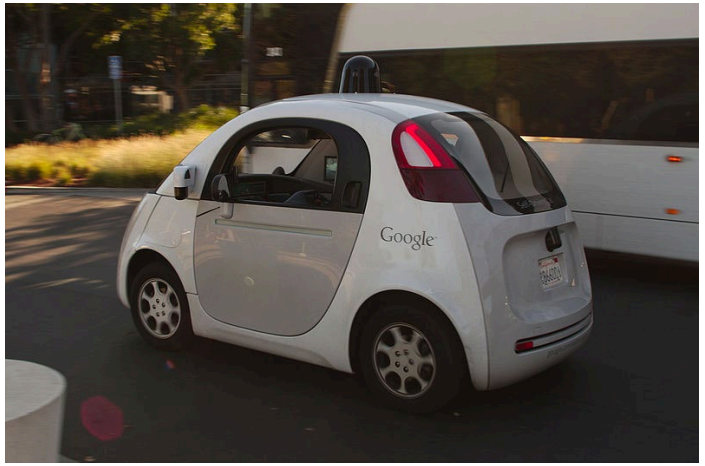
Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Automotive applications

- Use Cases:
  - Collision warning
  - Parking aid
  - Road sign detection
  - Autonomous cars
  - …







Source: Wikimedia, Wikipedia

Cinema

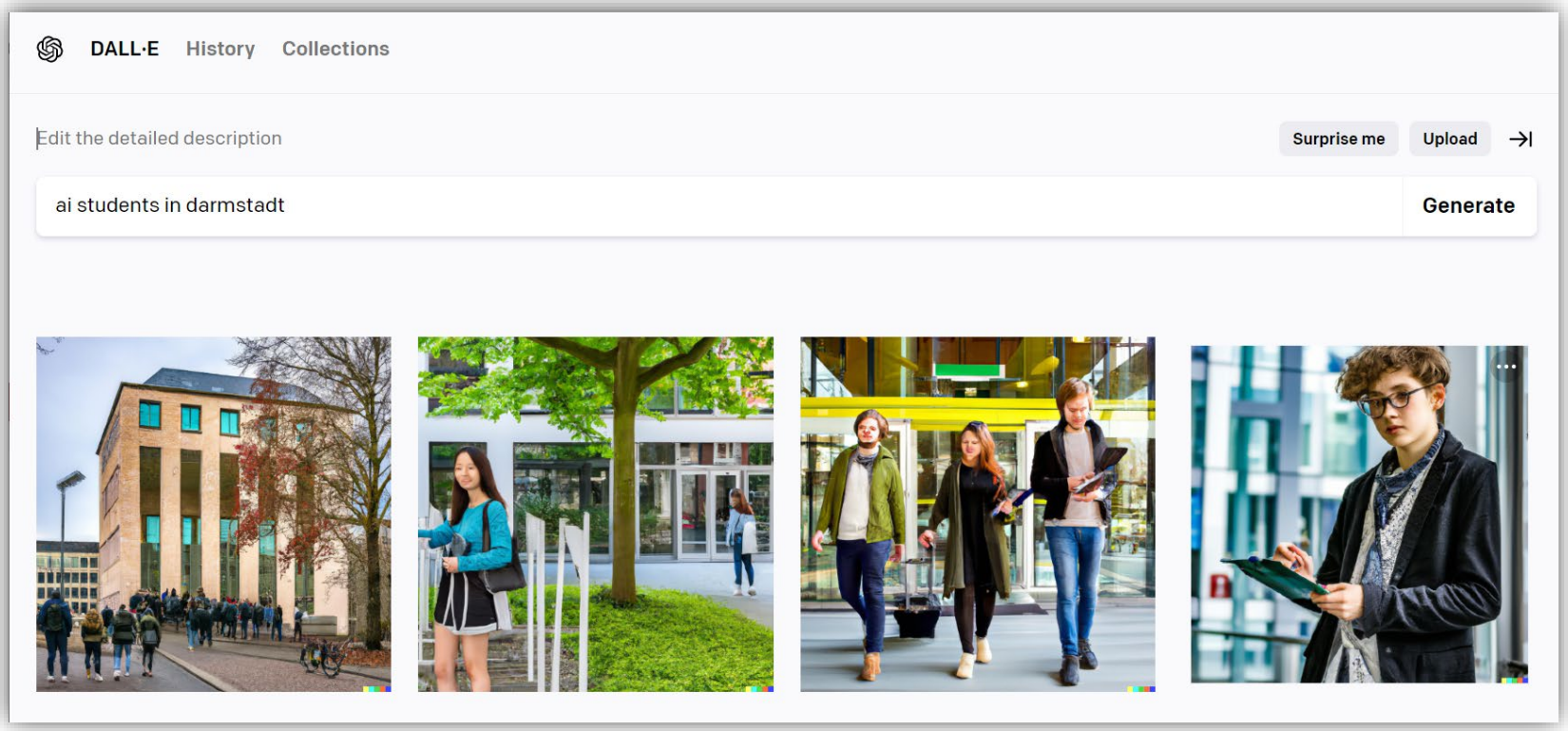


- Use Cases:
  - Generation of images scenes in animated movies
  - Motion capturing
  - …



Source: Wikimedia, Wikipedia

Image generation



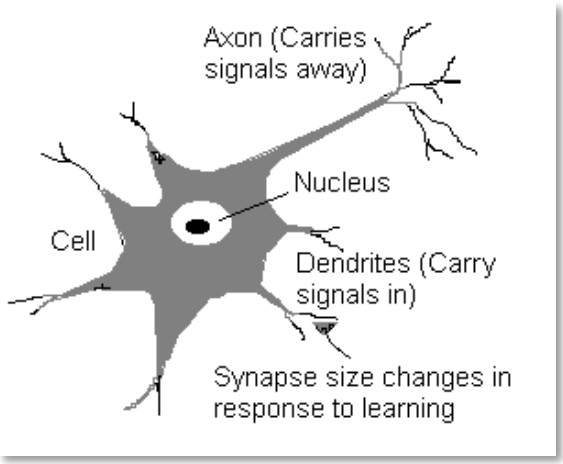
https://labs.openai.com

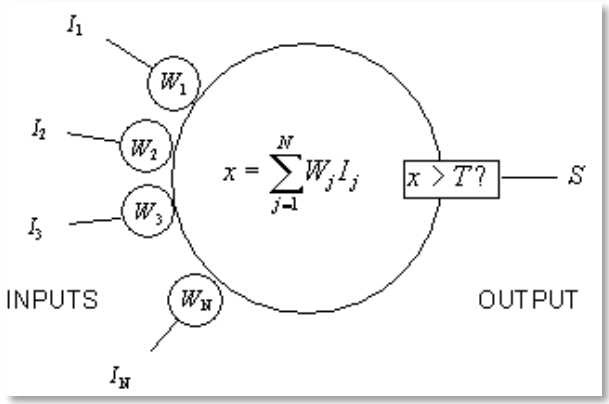
Agenda

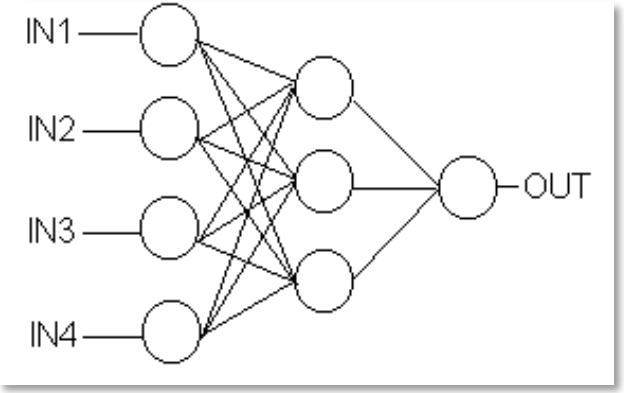
- Overview
- CV Applications
- Deep learning with CNN
- Challenge: Digit recognizer
- Quick check

Artificial Neural Networks (ANN)

- Inspired by brain / nervous system:
  - Neurons connected via dentrites
- Artificial Neuron:
  - Weighted inputs
  - Function, e.g., weighted sum
  - Filter, e.g, threshold → output
- Artificial Neural Network (ANN):
  - Input layer, output layer, and possibly intermediate layers of neurons





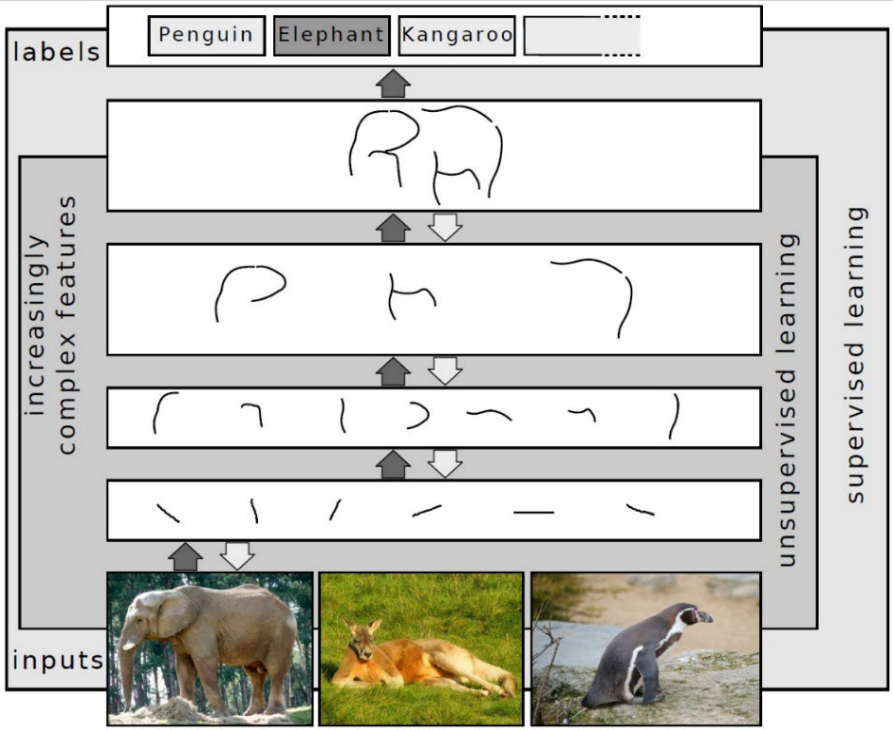


Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Source: Ivan Galkin, U. MASS Lowell (http://ulcar.uml.edu/~iag/CS/Intro-to-ANN.html)

Deep Learning (DL)

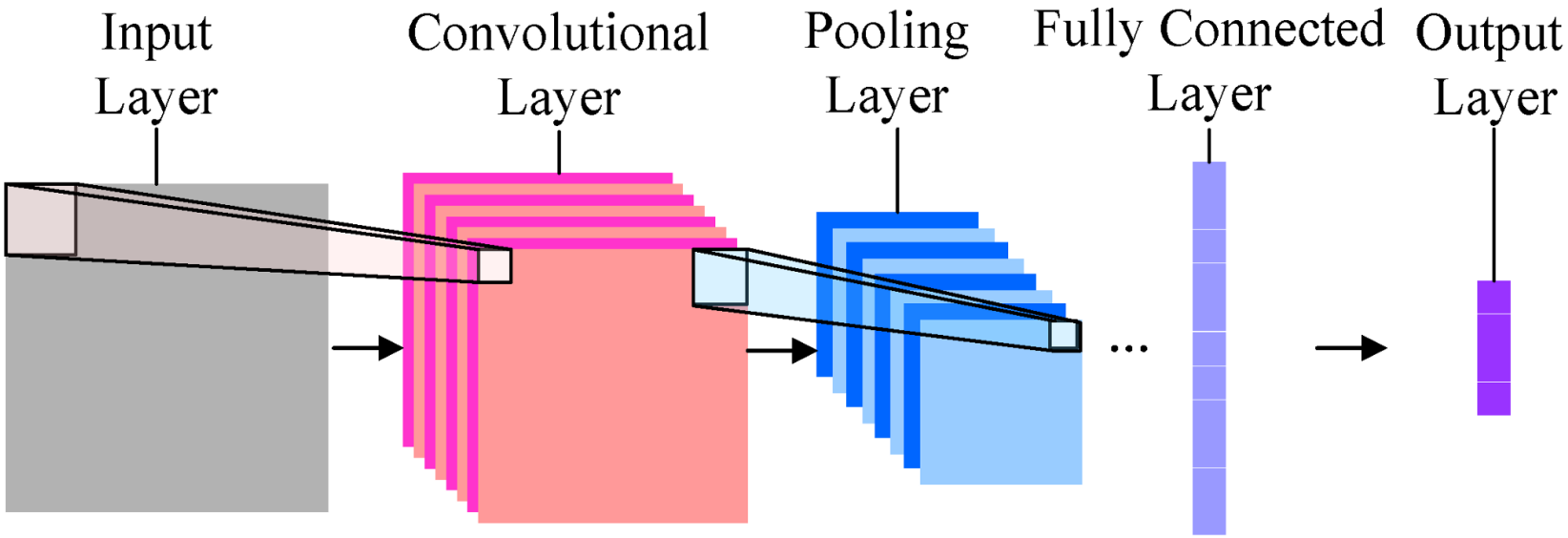
- Cascade of many layers in ANN
- Each successive layer uses the output from the previous layer as input
- From low-level (pixels) over medium level (shapes) to high level (classes)



https://commons.wikimedia.org/w/index.php?curid=82466022

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Convolutional Neural Networks (CNN)



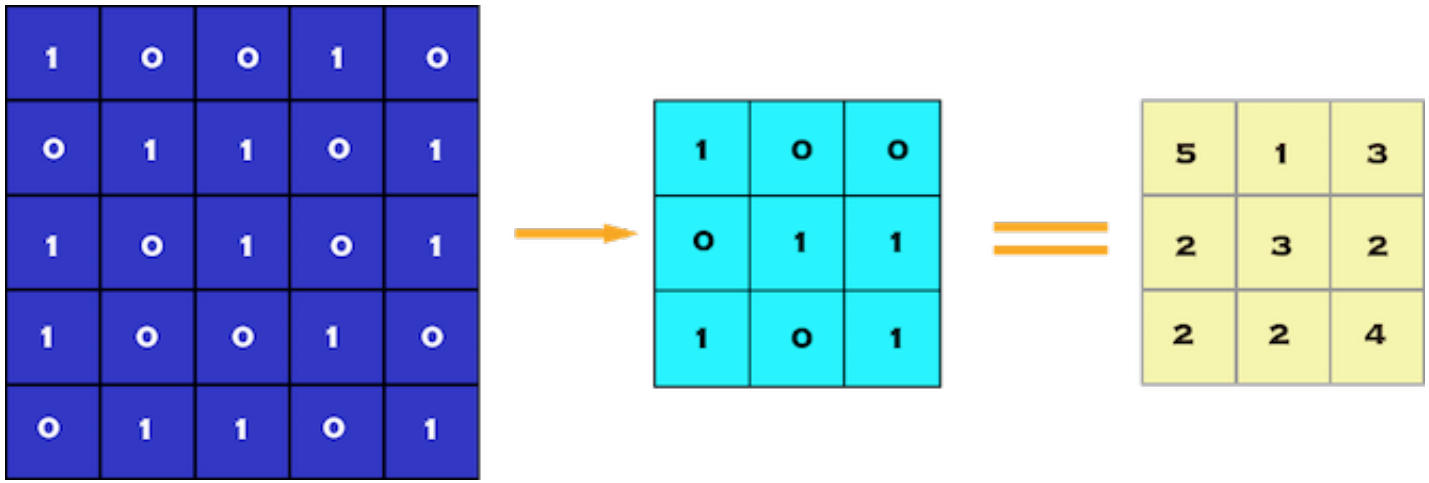
Input (low level) → Feature learning (intermediate level) → Classification → Output (high-level)

https://towardsdatascience.com/keras-transfer-learning-for-beginners-6c9b8b7143e

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Convolutional layers

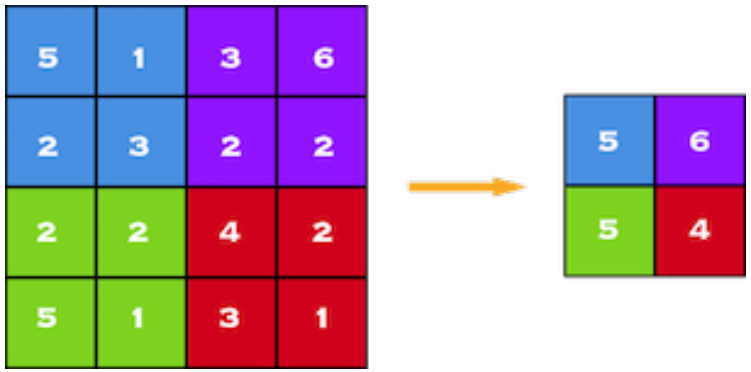
- Pixels are only related with the adjacent and close pixels
  → Preserve the relationship between different parts of an image
- Convolution: Filtering the image with a smaller pixel filter
  → Decreases the size of the image without losing the relationship between pixels



https://towardsdatascience.com/image-classification-in-10-minutes-with-mnist-dataset-54c35b77a38d

Pooling layers

- Select ranges of pixels and compute maximum, average, or sum values inside these pixels
- Example: Max pooling by 2x2

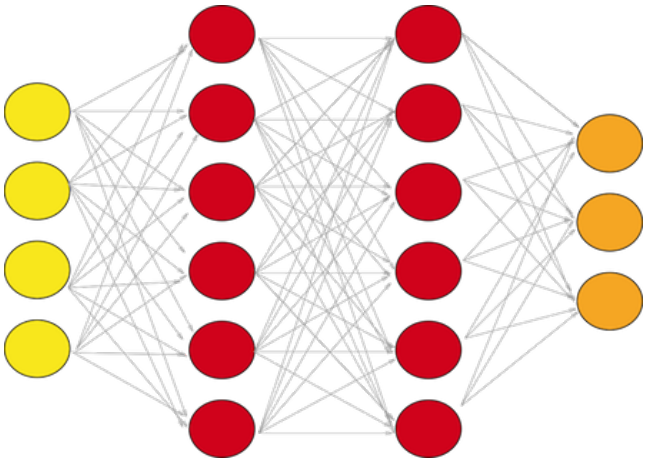


- Insert pooling layers after each convolution layer
  → Reduces the computational complexity

https://towardsdatascience.com/image-classification-in-10-minutes-with-mnist-dataset-54c35b77a38d

Fully connected layers

- Fully connected layer:
  Each neuron is connected with all other neurons
- Used for classifying images;
  as many output neurons as classes
- Example:



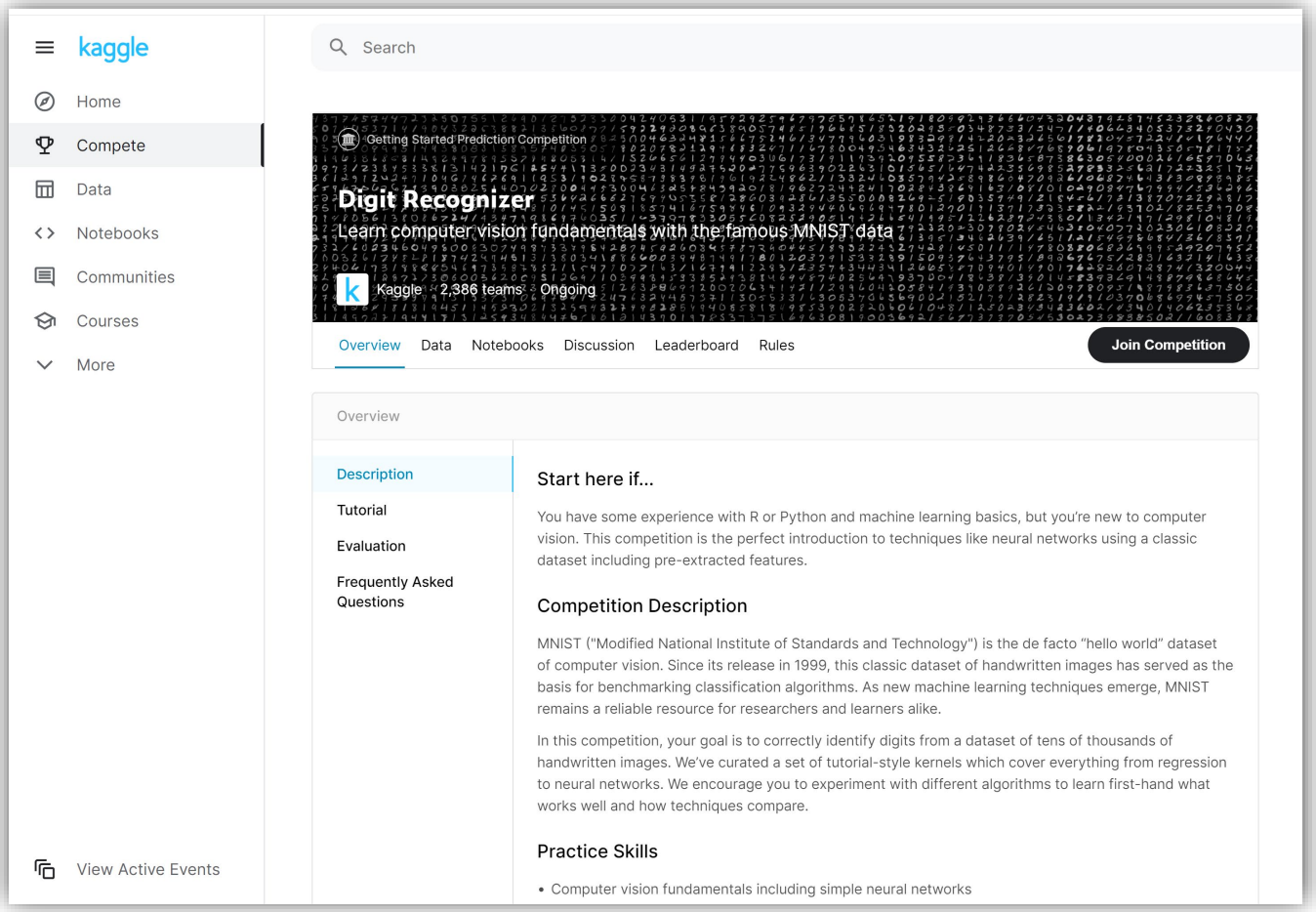
https://towardsdatascience.com/image-classification-in-10-minutes-with-mnist-dataset-54c35b77a38d

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Agenda

- Overview
- CV Applications
- Deep learning with CNN
- Challenge: Digit recognizer
- Quick check

Kaggle Challenge: Digit Recognizer



https://www.kaggle.com/c/digit-recognizer

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

The MNIST database of handwritten digits

- MNIST Database: http://yann.lecun.com/exdb/mnist/
- Task: Identify digit in image (OCR)
- Example input:



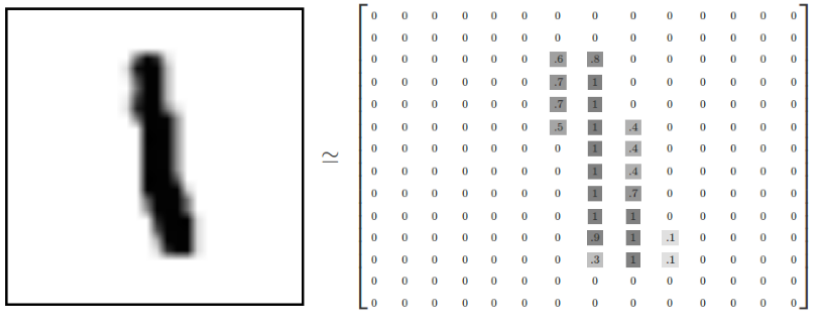
- Example output: 5 0 4 1

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

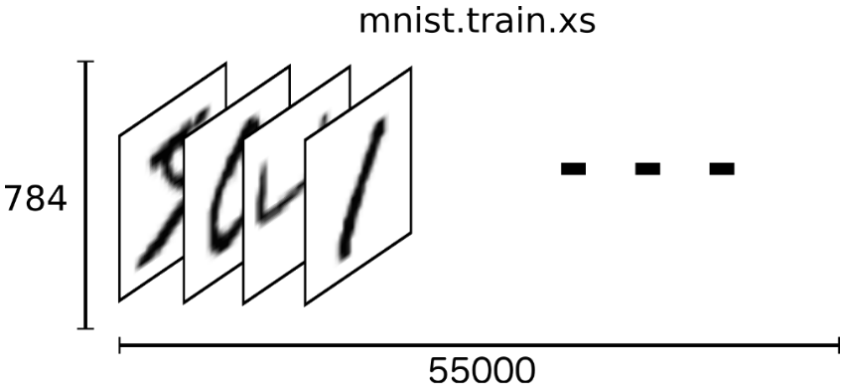
https://www.tensorflow.org/versions/r1.0/get_started/mnist/beginners

Representation of images

- Representation of individual image:
  - vector of 28x28 = 784 numbers



- Data set:
  - Array [55000, 784]



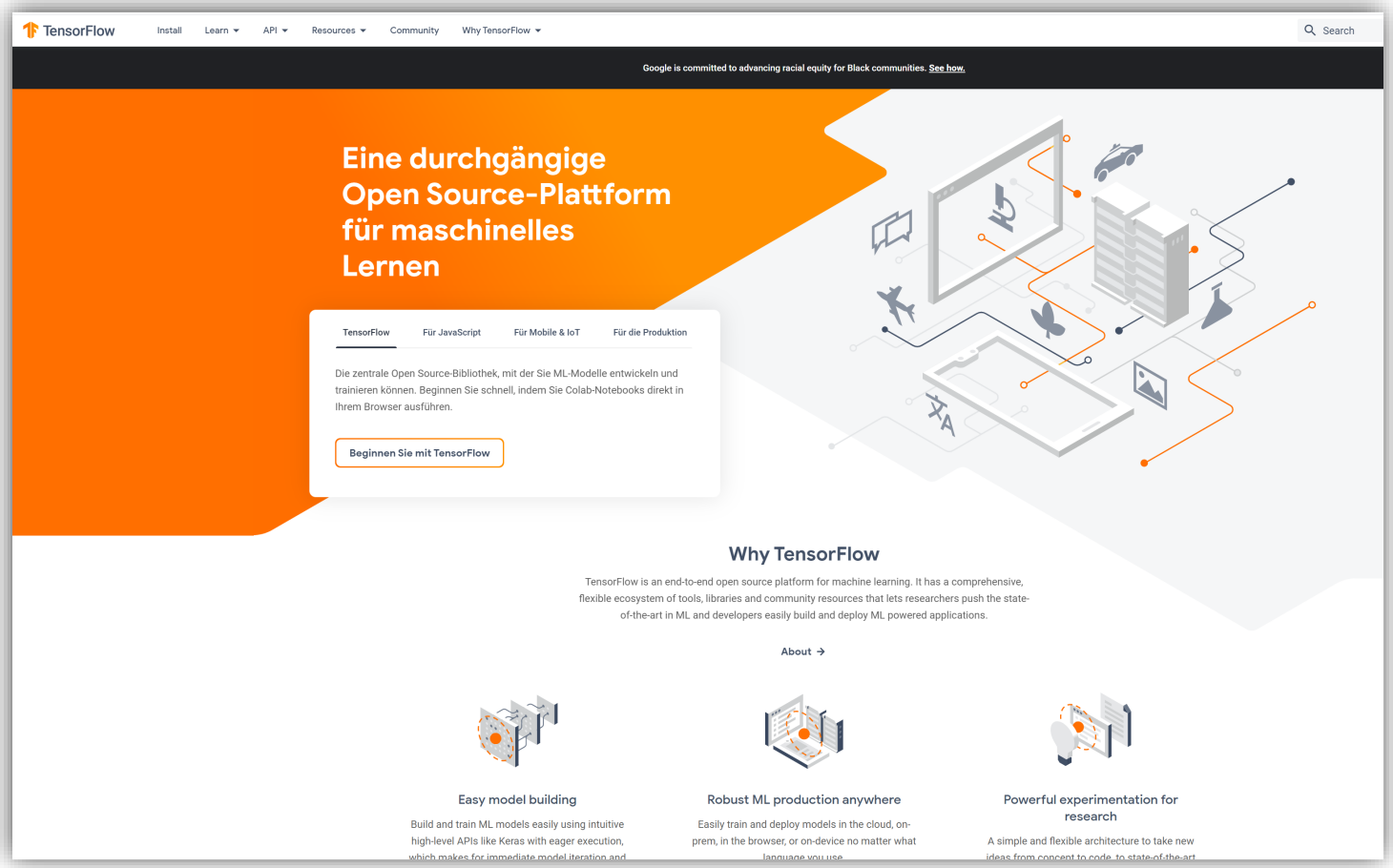
Data set representation

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	...	pixel774	pixel775	pixel776	pixel77
0	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0
3	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0

5 rows × 785 columns

https://www.kaggle.com/nitinbasantwani/digit-recognizer-using-basic-cnn-my-1st-notebook

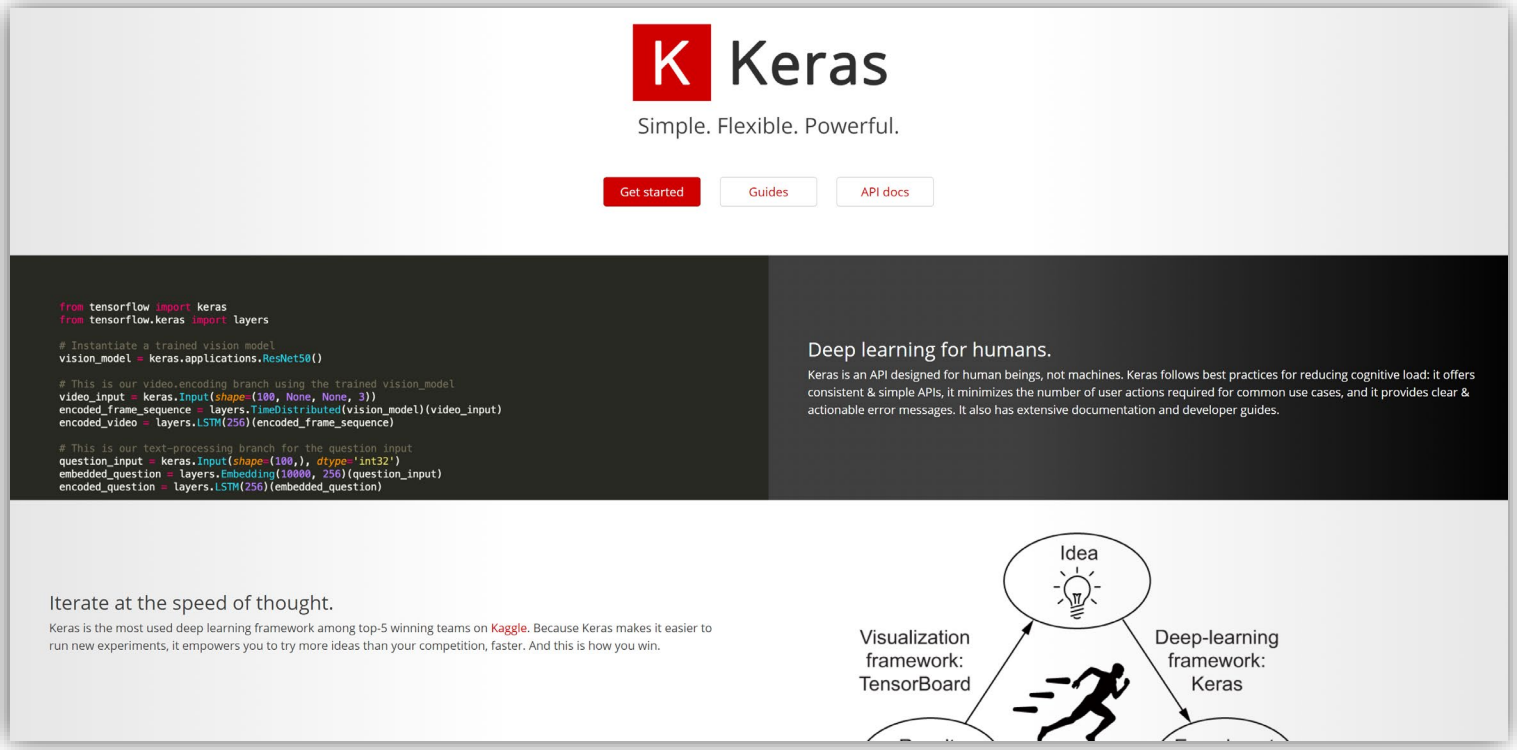
Tensorflow:
ML library by Google



https://www.tensorflow.org

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

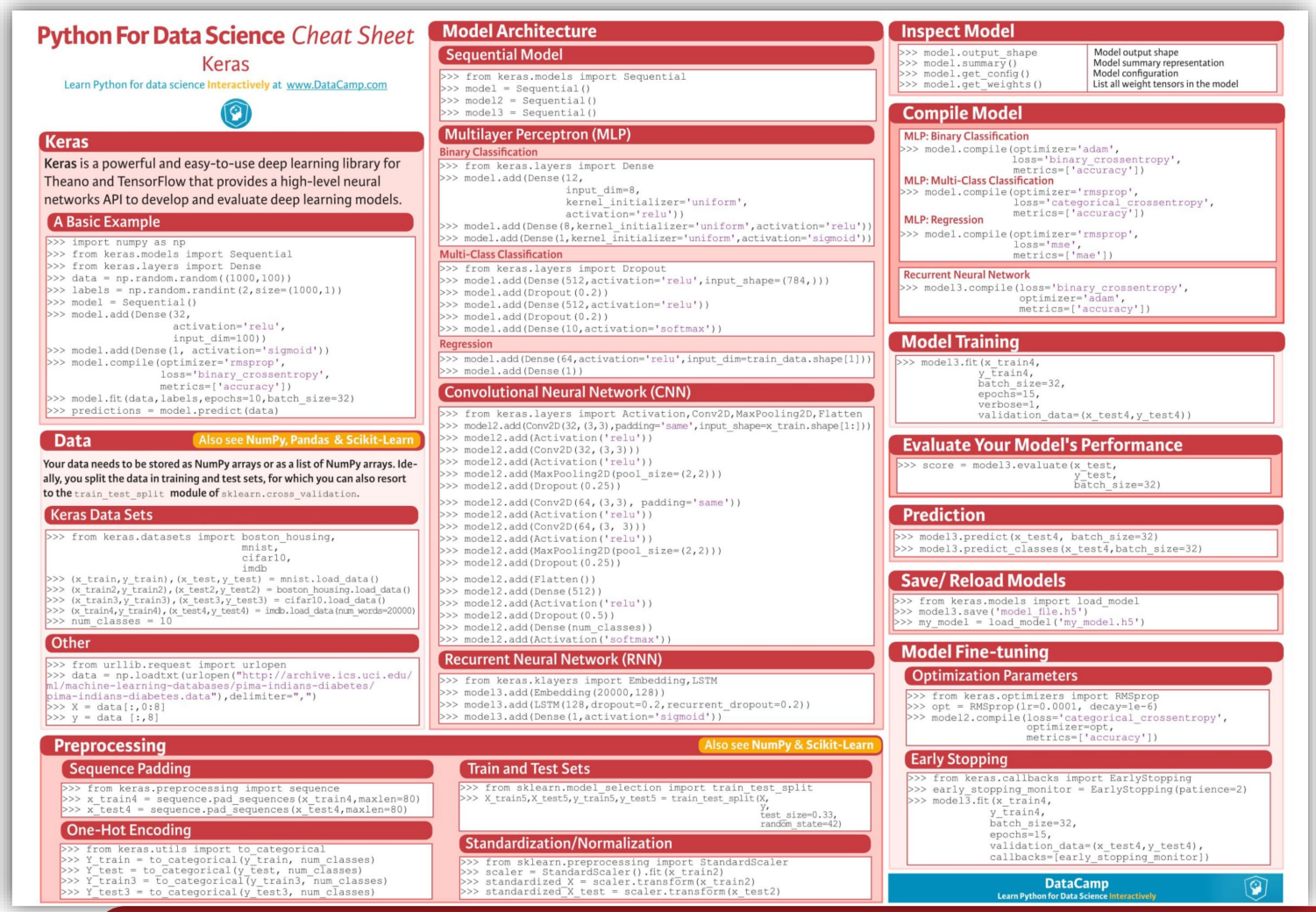
Keras:
Deep Learning API



https://keras.io

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Keras cheat sheet



Python For Data Science *Cheat Sheet*

Keras

Learn Python for data science Interactively at www.DataCamp.com

Keras

Keras is a powerful and easy-to-use deep learning library for Theano and TensorFlow that provides a high-level neural networks API to develop and evaluate deep learning models.

A Basic Example

```
>>> import numpy as np
>>> from keras.models import Sequential
>>> from keras.layers import Dense
>>> data = np.random.random((1000,100))
>>> labels = np.random.randint(2,size=(1000,1))
>>> model = Sequential()
>>> model.add(Dense(32,
                    activation='relu',
                    input_dim=100))
>>> model.add(Dense(1, activation='sigmoid'))
>>> model.compile(optimizer='rmsprop',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
>>> model.fit(data,labels,epochs=10,batch_size=32)
>>> predictions = model.predict(data)
```

Data

Also see NumPy, Pandas & Scikit-Learn

Your data needs to be stored as NumPy arrays or as a list of NumPy arrays. Ideally, you split the data in training and test sets, for which you can also resort to the `train_test_split` module of `sklearn.cross_validation`.

Keras Data Sets

```
>>> from keras.datasets import boston_housing,
                               mnist,
                               cifar10,
                               imdb

>>> (x_train,y_train),(x_test,y_test) = mnist.load_data()
>>> (x_train2,y_train2),(x_test2,y_test2) = boston_housing.load_data()
>>> (x_train3,y_train3),(x_test3,y_test3) = cifar10.load_data()
>>> (x_train4,y_train4),(x_test4,y_test4) = imdb.load_data(num_words=20000)
>>> num_classes = 10
```

Other

```
>>> from urllib.request import urlopen
>>> data = np.loadtxt(urlopen("http://archive.ics.uci.edu/
ml/machine-learning-databases/pima-indians-diabetes/
pima-indians-diabetes.data"),delimiter=",")
>>> X = data[:,0:8]
>>> y = data [:,8]
```

Model Architecture

Sequential Model

```
>>> from keras.models import Sequential
>>> model = Sequential()
>>> model2 = Sequential()
>>> model3 = Sequential()
```

Multilayer Perceptron (MLP)

Binary Classification

```
>>> from keras.layers import Dense
>>> model.add(Dense(12,
                    input_dim=8,
                    kernel_initializer='uniform',
                    activation='relu'))
>>> model.add(Dense(8,kernel_initializer='uniform',activation='relu'))
>>> model.add(Dense(1,kernel_initializer='uniform',activation='sigmoid'))
```

Multi-Class Classification

```
>>> from keras.layers import Dropout
>>> model.add(Dense(512,activation='relu',input_shape=(784,)))
>>> model.add(Dropout(0.2))
>>> model.add(Dense(512,activation='relu'))
>>> model.add(Dropout(0.2))
>>> model.add(Dense(10,activation='softmax'))
```

Regression

```
>>> model.add(Dense(64,activation='relu',input_dim=train_data.shape[1]))
>>> model.add(Dense(1))
```

Convolutional Neural Network (CNN)

```
>>> from keras.layers import Activation,Conv2D,MaxPooling2D,Flatten
>>> model2.add(Conv2D(32,(3,3),padding='same',input_shape=x_train.shape[1:]))
>>> model2.add(Activation('relu'))
>>> model2.add(Conv2D(32,(3,3)))
>>> model2.add(Activation('relu'))
>>> model2.add(MaxPooling2D(pool_size=(2,2)))
>>> model2.add(Dropout(0.25))

>>> model2.add(Conv2D(64,(3,3), padding='same'))
>>> model2.add(Activation('relu'))
>>> model2.add(Conv2D(64,(3, 3)))
>>> model2.add(Activation('relu'))
>>> model2.add(MaxPooling2D(pool_size=(2,2)))
>>> model2.add(Dropout(0.25))

>>> model2.add(Flatten())
>>> model2.add(Dense(512))
>>> model2.add(Activation('relu'))
>>> model2.add(Dropout(0.5))
>>> model2.add(Dense(num_classes))
>>> model2.add(Activation('softmax'))
```

Recurrent Neural Network (RNN)

```
>>> from keras.klayers import Embedding,LSTM
>>> model3.add(Embedding(20000,128))
>>> model3.add(LSTM(128,dropout=0.2,recurrent_dropout=0.2))
>>> model3.add(Dense(1,activation='sigmoid'))
```

Inspect Model

```
>>> model.output_shape          Model output shape
>>> model.summary()             Model summary representation
>>> model.get_config()          Model configuration
>>> model.get_weights()         List all weight tensors in the model
```

Compile Model

MLP: Binary Classification

```
>>> model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
```

MLP: Multi-Class Classification

```
>>> model.compile(optimizer='rmsprop',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
```

MLP: Regression

```
>>> model.compile(optimizer='rmsprop',
                  loss='mse',
                  metrics=['mae'])
```

Recurrent Neural Network

```
>>> model3.compile(loss='binary_crossentropy',
                   optimizer='adam',
                   metrics=['accuracy'])
```

Model Training

```
>>> model3.fit(x_train4,
               y_train4,
               batch_size=32,
               epochs=15,
               verbose=1,
               validation_data=(x_test4,y_test4))
```

Evaluate Your Model's Performance

```
>>> score = model3.evaluate(x_test,
                            y_test,
                            batch_size=32)
```

Prediction

```
>>> model3.predict(x_test4, batch_size=32)
>>> model3.predict_classes(x_test4,batch_size=32)
```

Save/ Reload Models

```
>>> from keras.models import load_model
>>> model3.save('model_file.h5')
>>> my_model = load_model('my_model.h5')
```

Model Fine-tuning

Optimization Parameters

```
>>> from keras.optimizers import RMSprop
>>> opt = RMSprop(lr=0.0001, decay=1e-6)
>>> model2.compile(loss='categorical_crossentropy',
                   optimizer=opt,
                   metrics=['accuracy'])
```

Early Stopping

```
>>> from keras.callbacks import EarlyStopping
>>> early_stopping_monitor = EarlyStopping(patience=2)
>>> model3.fit(x_train4,
               y_train4,
               batch_size=32,
               epochs=15,
               validation_data=(x_test4,y_test4),
               callbacks=[early_stopping_monitor])
```

Preprocessing

Also see NumPy & Scikit-Learn

Sequence Padding

```
>>> from keras.preprocessing import sequence
>>> x_train4 = sequence.pad_sequences(x_train4,maxlen=80)
>>> x_test4 = sequence.pad_sequences(x_test4,maxlen=80)
```

One-Hot Encoding

```
>>> from keras.utils import to_categorical
>>> Y_train = to_categorical(y_train, num_classes)
>>> Y_test = to_categorical(y_test, num_classes)
>>> Y_train3 = to_categorical(y_train3, num_classes)
>>> Y_test3 = to_categorical(y_test3, num_classes)
```

Train and Test Sets

```
>>> from sklearn.model_selection import train_test_split
>>> X_train5,X_test5,y_train5,y_test5 = train_test_split(X,
                                                         y,
                                                         test_size=0.33,
                                                         random_state=42)
```

Standardization/Normalization

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler().fit(x_train2)
>>> standardized_X = scaler.transform(x_train2)
>>> standardized_X_test = scaler.transform(x_test2)
```

DataCamp
Learn Python for Data Science Interactively

https://medium.com/@_moazzemhossain/machine-learning-cheat-sheets-for-ml-engineers-data-scientist-d44a43f65979

Loading data

```
# Loading data
train = pd.read_csv('data/train.csv')
test = pd.read_csv('data/test.csv')
```

Preprocessing

```
# Split features and labels in training set
X = train.drop(labels = ['label'], axis = 1)
y = train['label']

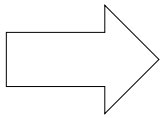
# One hot encode labels
Y = to_categorical(y, num_classes = 10)

# Normalize pixel data
X = X/255.0

# Reshape image in 3 dimensions (height = 28px, width = 28px , canal = 1)
X = X.values.reshape(-1, 28, 28, 1)
```

One-Hot Encoding

Label
3
5
9
1
0
6
5



0	1	2	3	4	5	6	7	8	9
0	0	0	**1**	0	0	0	0	0	0
0	0	0	0	0	**1**	0	0	0	0
0	0	0	0	0	0	0	0	0	**1**
0	**1**	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	**1**	0	0	0
0	0	0	0	0	**1**	0	0	0	0

Defining the model

ANN Topology: Sequence of layers

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.optimizers import RMSprop

model = Sequential([
  Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
  MaxPooling2D(2, 2),
  Conv2D(64, (3,3), activation='relu'),
  MaxPooling2D(2,2),
  Flatten(),
  Dense(512, activation='relu'),
  Dense(10, activation='softmax')
])
```

2 convolutional and 2 pooling layers

Activation function: computes outputs of individual neurons

Flatten into a single feature vector

2 fully connected layers

Training the model

Optimizer used for optimizing metrics (gradient descent)

Loss function computes the error, i.e., the difference between actual output and predicted output

Metrics: what needs to be optimized

```
# Compiling model
model.compile(optimizer=RMSprop(), loss='categorical_crossentropy', metrics=['accuracy'])

# Fitting model
history = model.fit(X, Y, batch_size=100, epochs = 1)
```

Batch size: number of training examples utilized in one iteration of the optimization process

Number of epochs: how often the entire training data set is used for optimization; larger numbers will increase accuracy but also training runtime

Evaluating the model

Training ANN can result in long training runtime. Cross-validation may take a long time. Alternative: train_test_split() allows splitting data in training set and test set once.

```
# Split the data in training and test set
X_train, X_test, Y_train, Y_test = train_test_split(X, Y)

# TODO: train and evaluate
```

Generating submission file

	A	B
1	ImageId	Label
2	1	2
3	2	0
4	3	9
5	4	9
6	5	3
7	6	7
8	7	0
9	8	3
10	9	0
11	10	3
12	11	5
13	12	7
14	13	4
15	14	0
16	15	4
17	16	3
18	17	3
19	18	1
20	19	9
21	20	0
22	21	9
23	22	1

Test data needs to be pre-processed like training data

```
# Predict results
results = model.predict(test)
# One hot decoding: Select the index with the maximum probability
results = np.argmax(results, axis = 1)
# Add columns ImageID and Label
results = pd.Series(results, name="Label")
submission = pd.concat([pd.Series(range(1, len(results)+1), name = "ImageId"), results], axis = 1)
# Generate CSV file
submission.to_csv('data/submission.csv', index=False)
```

Agenda

- Overview
- CV Applications
- Deep learning with CNN
- Challenge: Digit recognizer
- Quick check

Quick check: Computer Vision

1. Name applications of CV
2. What is an ANN?
3. What is deep learning?
4. What is a CNN?
5. How are images in the MNIST database represented?
6. What is Keras and Tensorflow? How to use it?
7. Explain one-hot-encoding